Week 45

v 1.0.3

Table of contents

1 Python: Introduction to plotting 1
2 Average properties of amino acids and proteins. 6
3 UV spectrum of apo- and holo-myoglobin 10
4 Proteins in blood plasma. 13
5 Extinction coefficient of human myoglobin 16
6 Tryptophan absorption and fluorescence 21
7 The fluorescent protein “mCherry”. 25
8 Python: Introduction to curve fitting 28

```
try:
    import fysisk_biokemi
    print("Already installed")
except ImportError:
    %pip install -q "fysisk_biokemi[pyodide] @ https://au-mbg.github.io/fysisk-biokemi/
wheels/fysisk_biokemi-0.2.2-py3-none-any.whl"
```

---

1 Python: Introduction to plotting

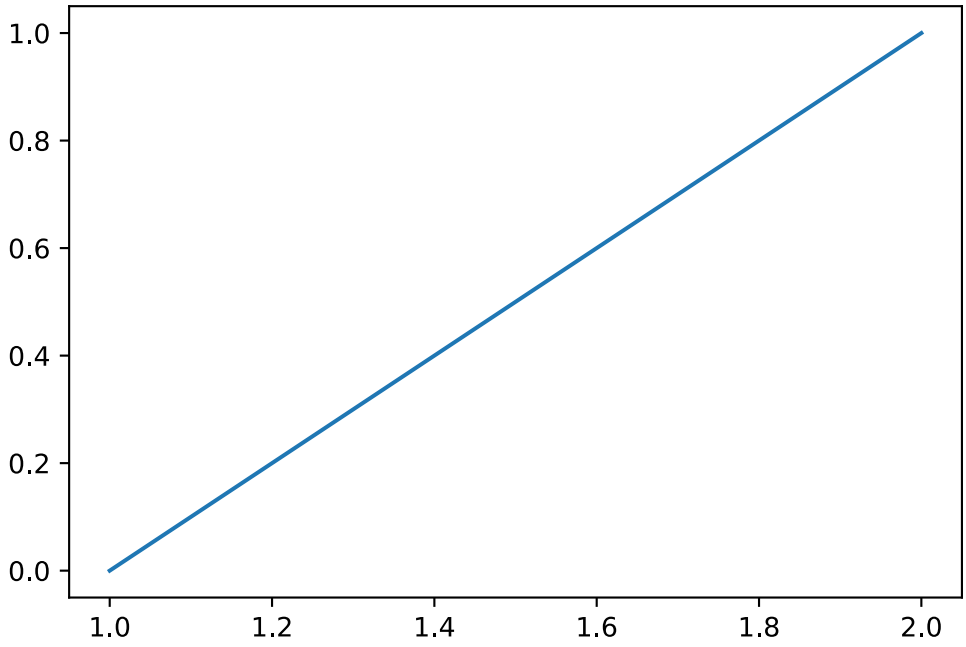
```
import numpy as np
import matplotlib.pyplot as plt
```

There are many ways to plot, we will be using the `matplotlib` library - but the concepts are similar across most ways of plotting.

> **i** Note
>
> A library is a collection of ready-made code that helps you perform tasks in Python, so you don’t have to write everything yourself.

We can make a plot of a straight line between two points (x_1, y_1) and (x_2, y_2) like so;

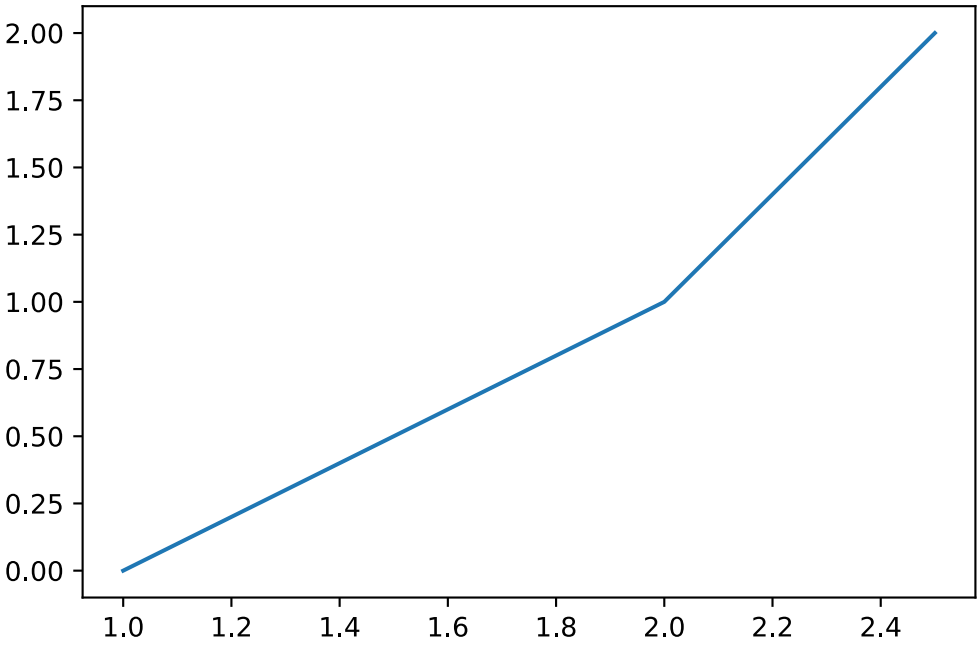
```
x = np.array([1, 2]) # This array contains all the x-coordinates
y = np.array([0, 1]) # And this contains all the y-coordinates.
fig, ax = plt.subplots(figsize=(6, 4)) # 6,4 defines the size of the plot
ax.plot(x, y)
```



> **Note**
>
> `ax` is a special type (like `int`, `float`, `str`), and is the area where you create your plot. You can think of it as the canvas you draw on.

If we want to connect to a third point (x_3, y_3) we would instead write

```
x = np.array([1, 2, 2.5])
y = np.array([0, 1, 2])
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y)
```



Notice that with three sets of points we plot **two** line segments.

(a) Trying to plot a square

Make a plot with lines between the following 4

- $P_1 = (0, 0)$
- $P_2 = (1, 0)$
- $P_3 = (1, 1)$
- $P_4 = (0, 1)$

Start by defining an array for both `x` and `y`

```
x = np.array([...]) # Replace ... with your code
# Do the equivalent for y
```

Now use `ax.plot` to make the plot

```
fig, ax = plt.subplots(figsize=(5, 5))
... # Replace with your code.
ax.axis('equal')
ax.set_xlim([-0.5, 1.5]) # Tells matplotlib to show us the region between -0.5 and 1.5 on
the x-axis.
ax.set_ylim([-0.5, 1.5])
```

Consider the following;

- Why is it not a square?
- How many points did you use?
- How many line segments were plotted?

(b) Actually plotting a square

In order to actually plot a square you will need to update `x` and `y`, they both need to contain five numbers such that four line segments are plotted.

```
x = np.array([...]) # Replace ... with your code
# Do the equivalent for y
```

Copy your code for making the plot from the previous exercise to the cell below

```
... # Put your copied code here.
```

(c) Plotting a function

Often the data we plot can broadly be thought of as originating from a function, such as:

$$y = f(x)$$

Where f is the function, this might for example be an experiment that produces some output y given some input x or it might be a traditional mathematical function like $y = x^2$. Plotting this type of data is exactly the same as plotting a square - it just usually consists of many more data points resulting in many line segments producing a smooth looking curve.

Make a plot of the function

$$y = \mathrm{e}^x$$

> **Note**
>
> The exponential function can be calculated using `np.exp` - likewise with the logarithm `np.log`.

```
x = np.linspace(0, 5, 50) # Makes 50 points uniformly between 0 and 5.
y = ... # Your code here.

fig, ax = plt.subplots(figsize=(6, 4))
... # Your code to plot here
```

Once you've made the plot try changing the number of points in `x` by changing the last argument in the call to `np.linspace`.

- What happens if you have very few points?

> **Tip**
>
> Remember that `np.exp` is used to calculate the exponential function.

(d) Plotting multiple functions

Often we want to plot multiple functions in the same figure as it enables us to compare them. Luckily, this is quite simple!

Plot the function

$$y = \mathrm{e}^{ax}$$

With $a = \left[1, \cfrac{1}{2}, 1.2\right]$ in the same plot producing three curves.

```
x = np.linspace(0, 5, 50) # Make 50 points uniformly between 0 and 5.
y1 = ...
y2 = ...
y3 = ...

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y1)
ax.plot(x, y2) # To plot a second curve we just use ax.plot again
... # Plot the the third curve.
```

(e) Customizing plots

We will generally need to customize plots a little bit, for example the plot from the previous exercise doesn't have a label for the axis, there's no way of telling which curve is which and it has no title.

To add labels to the x and y axis we use the two functions

- `ax.set_xlabel('String with the name of the x-axis')`
- `ax.set_ylabel('String with the name of the y-axis')`

Adding information about the curves we plot can be done by also giving them a label, that is done as an argument to the `ax.plot` call, like so

```
fig, ax = plt.subplots()

ax.plot(x, y, label='Information about the curve.')
ax.legend()
ax.set_title('Title of plot')
```

Line 3

The `label` argument is given when the curve is plotted.

Line 4

`ax.legend()` is called to tell matplotlib to show the a 'legend' containing the `label` of all the plots that have been given one.

Line 5

Sets the title of the plot

Copy your code from the previous exercise and add labels for both the x and y axis aswell as each curve, for example you can label them according to the value of the the parameter a like `a = 1` etc.

```
... # Put some code here and try customizing it.
```

```
Ellipsis
```

> **Note**
>
> There are many other customization options that can be added in the same way as the `label`, that is
>
> ```
> ax.plot(x, y, option_name=option_value)
> ```
>
> Some useful examples are
>
> - `linestyle`: Controls if the plot is made using a full, dashed or dotted line using `-`, `--` and `:` respectively. So `ax.plot(..., ..., linestyle = ":")` produces a dotted line.
> - `color`: Controls the color of the line - valid options are listed here https://matplotlib.org/stable/gallery/color/named_colors.html
> - `linewidth`: Sets the width of the line - can be any number.
> - `alpha`: Controls the transparency of the plot `0` being fully transparent and `1` being fully visible.
>
> You can try some of these options for the plot you've made above if you want to. There are *many* other ways of customizing plots for different situations. The matplotlib gallery shows a number of them.

(f) Other types of plots: Scatter

Sometimes we don't want to connect each point with a line segment but just show the points in a *scatter* plot.

The next cell makes some data

```
x = np.random.uniform(low=0, high=10, size=100)
y = 2*x + np.random.normal(scale=2, size=100)
```

Try plotting it like for the previous exercises

```
# Put your code to quickly plot in this cell - no customization needed!
```

You will see a very strange plot like when you scribble out a word on a piece of paper. Clearly a line plot is not a very good way of showing this data! There are, at least, two ways of making a scatter plot

- `ax.plot(x, y, 'o')`: Quick and dirty way of just showing the points and not the lines.
- `ax.scatter(x, y)`: Function specifically for making these types of plots - with its own set of customization arguments.

Try either, or both, of these ways in the cell below

```
fig, ax = plt.subplots()

# Your code here.
```

---

2 Average properties of amino acids and proteins.

```
import pandas as pd
import numpy as np
from IPython.display import display
from pathlib import Path
```

In the accompanying Excel file (`averag-prope-amino-acids.xlsx`), you will find a table that contains the molecular weight of the 20 common amino acid residues, i.e. their weight as residues in a peptide chain. Additionally, you will find their relative frequency in E. coli proteins, where a frequency of 0.01 means that this residue constitutes 1 % of the residues in a protein.

(a) Load the data file.

Load the `averag-prope-amino-acids.xlsx` dataset

Use the cell below to load and display the dataset.

> Need help loading the data?
>
> The correct pandas function is already provided. You only need to set `data_file_name`:
>
> 1. Download the file named above from the Datasets page.
> 2. Upload the file to JupyterLite.
> 3. Replace `...` with the file's name in quotation marks.

Insert the name of your file in the code cell below to read it into your notebook

```
data_file_name = ... # Write the path to the dataset file
df = pd.read_excel(data_file_name)
display(df)
```

(b) Average molecular weight

Calculate the average molecular weight of a residue in a protein. To do this our procedure will be as follows

1. First calculate the residue weight times its frequency in proteins for each residue.
2. Sum these together to get the average molecular weight.

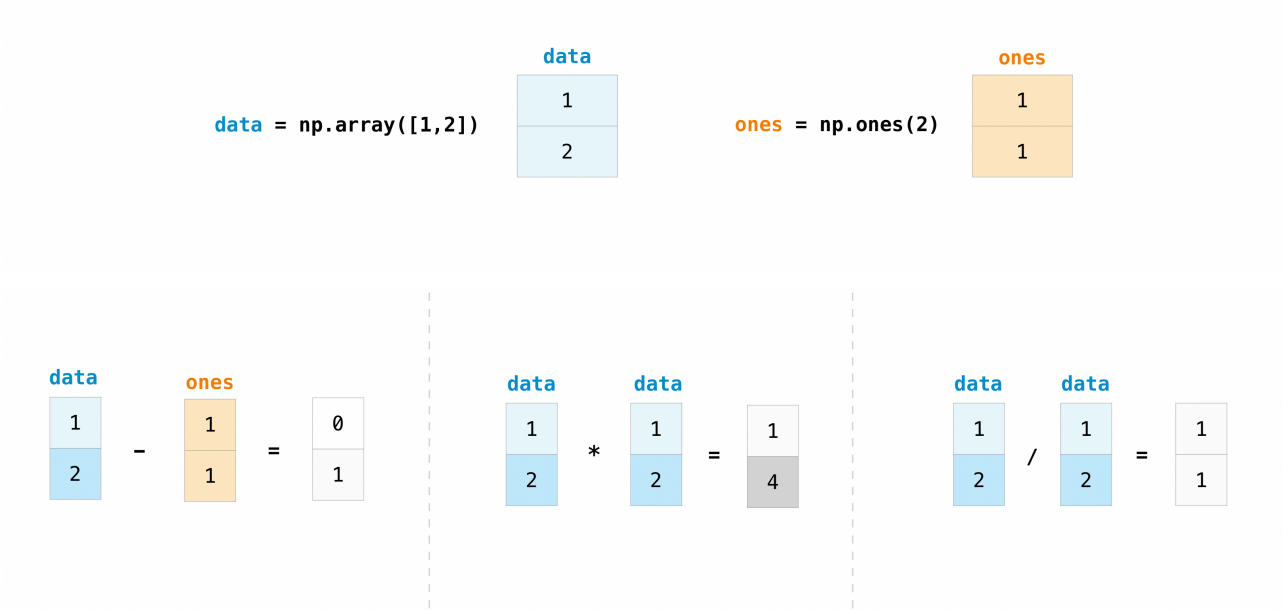
In the cell below finish the calculation of `weight_times_freq` by extracting the `"MW of AA residue"`-column and `"Frequency in proteins"` and multiplying them together.

Tip

You can index in the dataframe by using the column name, for example to get the `"MW of AA residue"`-column you would do

```
col = df["MW of AA residue"]
```

Arrays also allows us to operate on every element in the array at the same time, so arrays can be added, subtracted, multiplied, divided, etc, see for example this figure




```
weight_times_freq = ...
```

Tip

You can use `np.sum` to sum all values in an array. For example

```
array = np.array([1, 2, 3, 4, 5])
sum_of_array = np.sum(array) # Gives 15
```

```
average_mw = ...
unit_average_mw = "..."
print(f"{average_mw = :.3f} {unit_average_mw}")
```

> **Note**
>
> The syntax `f"{average_mw = :.3f}"` is just a way of printing the value with a nicer format, in this case we print the value to 3 decimal places. In Python these are called f-strings, you don't need to understand the details at the moment.

(c) Weight of 300-residue protein

What would the molecular weight of a 300-residue protein most likely be, if you did not know its sequence?

```
mw_300 = ...
unit_mw_300 = "..."
print(f"{mw_300 = :.3f} {unit_mw_300}")
```

In many projects, you will be working with a mixture of proteins. This could for example be a cell lysate or a biological fluid for protein abundance analysis, or the early stages of a protein purification process. In these situations, you cannot work with a molecule specific extinction coefficient. Instead, we would use the average values, which we will determine below.

(d) Average concentration of residues

In many cases, we work with mixtures of proteins that do not have a defined molar extinctio coefficient. Instead, we work with an average 'mass extinction coefficient', which we will deduce here.

Calculate the average concentration of amino acid residues in a protein mixture at 1 mg/mL.

```
c_residue_avg = ...
unit_c_residue_avg = "..."
print(f"{c_residue_avg = :3.3f} {unit_c_residue_avg}")
```

(e) Absorbance of an average mixture of residues

Calculate the absorbance from such a mixture under the assumption that only Trp and Tyr contribute, and that the path length is 1 cm.

```
freq = df.set_index("Name")["Frequency in proteins"]
f_trp = freq["Tryptophan (Trp/W)"]
f_tyr = freq["Tyrosine (Tyr/Y)"]
c_trp = ... # Erstat med din kode.
c_tyr = ... # Erstat med din kode.
unit_c_trp_tyr = "..."

print(f"{c_trp = :3.5f} {unit_c_trp_tyr}")
print(f"{c_tyr = :3.5f} {unit_c_trp_tyr}")
```

```
A280_1mg_pr_ml = ... # Erstat med din kode.
unit_A280_1mg_pr_ml = "..."
print(f"{A280_1mg_pr_ml = :3.3f} {unit_A280_1mg_pr_ml}")
```

In the compendium, we mentioned a "rule of thumb" that 1 mg/mL protein at a path length of 1 cm has an A280 = 1. How well does this compare to your calculated value?

(f) Absorbance → concentration

For a cell lysate, you measure and absorbance of 0.78 at a path length of 0.5 cm. What is the protein concentration?

```
# Set known values:
A = ... # Unitless
l = ... # cm

# Calculate concentration:
mass_conc = ...
unit_mass_conc = "..."
print(f"Protein concentration = {mass_conc:.3f} {unit_mass_conc}")
```

---

3 UV spectrum of apo- and holo-myoglobin

```
import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display
```

In this exercise we will analyze the spectra of apo- and holo-myoglobin. The dataset is given in `uv-spec-apo-holo-myo.csv`.

> Need help loading the data?
>
> The correct pandas function is already provided. You only need to set `data_file_path`:
>
> 1. Download the file named above from the Datasets page.
> 2. Upload the file to JupyterLite.
> 3. Replace `...` with the file's name in quotation marks.

Insert the name of your file in the code cell below to read it into your notebook

```
data_file_name = ... # Write the path to the dataset file
df = pd.read_csv(data_file_name)
display(df)
```

(b) Plot the spectra

Tip

To plot we use the `matplotlib` package. Plots are generally just straight lines connecting points, with enough points we get a smooth looking figure.

For example, to plot a line connecting three datapoints

```
fig, ax = plt.subplots()

x = np.array([1, 2, 3])
y = np.array([1, 4, 9])
ax.plot(x, y)
plt.show()
```

Line 1

This creates a `fig` and an `ax`, the `ax`-object is the box where our plot is created.

Line 3

Defines the data to put on the x-axis.

Line 5

Does the actual plotting

Line 6

Shows the figure

There are *many* ways of customizing plots, you will see different ones in the exercises, but by no means all of them - if you are interested you can find more information on the matplotlib documentation.

You don't have to worry about the `NaN` values in the dataset when plotting, matplotlib just skips plotting that line segment.

Using `matplotlib` plot each spectrum in the same figure as line plots.

```
fig, ax = plt.subplots()

# This selects the 'holo_absorbance'-column and plots it
ax.plot(df['wavelength'], df['holo_absorbance'], label='Holo')

# Copy the line above and edit it to also plot the Apo absorbance
... # Your code here

ax.set_ylabel('Absorbance (AU)')
ax.set_xlabel('Wavelength (nm)')
ax.legend()
```

(c) Apo & Holo

Based on the spectra explain what ApoMb and HoloMb represent?

You have learned that pure proteins without any UV/Vis-absorbing prosthetic groups bound, have basically no absorbance at wavelengths above λ>320 nm. Nevertheless ApoMb still show some absorbance above 320 nm. In this case, it can explained by the fact that ApoMb was generated from HoloMb by a procedure that will not be explained here.

(d) Absorption above 320 nm.

Explain why ApoMb in the spectrum above absorbs light at λ>320 nm.

(e) Efficiency estimate.

Give a rough estimate of the efficiency of the chosen procedure of heme-group removal. You may wish to look at the numerical values in the `DataFrame`.

(f) Isobestic points

Are there any isobestic points between the two spectra?

To get a better understanding of the causes for the different spectra, you can compare to litterature. The figure below shows the absorbance spectra of three states of myoglobin.

$$\mathrm{MbFe(II)O_2} \Rightarrow \text{oxyMb}$$

$$\mathrm{MbFe(III)} \Rightarrow \text{metMb}$$

$$\mathrm{MbFe(II)} \Rightarrow \text{deoxyMb}$$

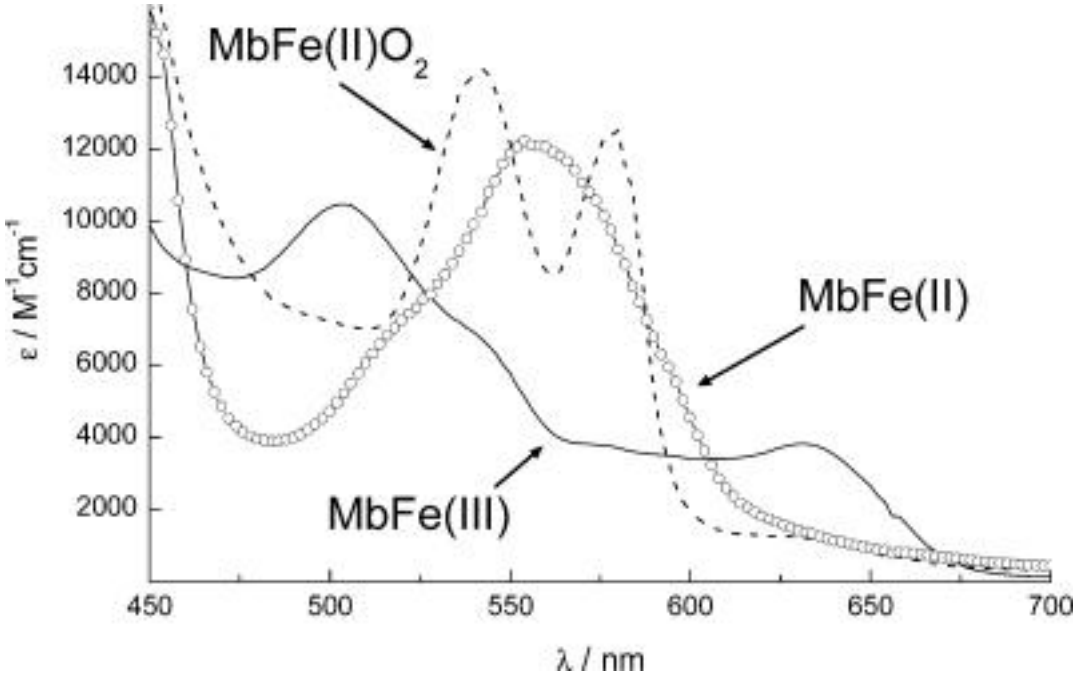


Figure 1: Absorbance spectra of different states of myoglobin. $\mathrm{MbFe(II)O_2}$ is "oxyMb", $\mathrm{MbFe(III)}$ is "metMb" and $\mathrm{MbFe(II)}$ is "deoxyMb". Taken from Barham et al (2010); ChemRev; 110 (4); pp2313-2365

(g) Aged myoglobin

metMb (MbFe(III)) is normally described as 'aged' myoglobin. What does this mean in terms of the bound iron?

(h) Change in absorbance

Give a qualitative explanation to the observed change in absorbance of metMb compared to fresh oxy-/deoxyMb?

(i) Type of shift

Is the spectral difference from deoxyMb to metMb a redshift or a blueshift?

(j) Experiment design

You would like to set up an experiment where the absorbance of myoglobin at defined wavelength(s) should be used to measure the level of O_2 binding. Sketch how the absorbance spectra would look like when going from deoxyMb and continuously increasing the concentration of O_2 (draw this for five different O_2 concentrations going from pure deoxyMb to pure oxyMb)

---

4 Proteins in blood plasma.

```
import pandas as pd
from IPython.display import display
from pathlib import Path
```

In this exercise we will learn how Python is excellent for handling datasets with many data points and how it can be used to apply the same procedure to all the data points at once.

A researcher wants to determine the concentration of two proteins in blood plasma that is suspected to be involved in development of an autoimmune disease. 500 patients and 500 healthy individuals were included in the study and absorbance measurements of the two purified proteins from all blood plasma samples were measured at 280 nm. The molecular weight and extinction coefficients of the two proteins are given in the table below.

Protein	M_w [kDa]	ϵ [$M^{-1}cm^{-1}$]
1	130	180000
2	57	80000

(a) Load the dataset

Load the dataset as a dataframe from the file `protei-blood-plasma.xlsx` with pandas. Set `data_file_name` to the path of the dataset file.

> Need help loading the data?
>
> The correct pandas function is already provided. You only need to set `data_file_path`:
>
> 1. Download the file named above from the Datasets page.
> 2. Upload the file to JupyterLite.
> 3. Replace `...` with the file's name in quotation marks.

Insert the name of your file in the codecell below to read it into yout notebook

```
data_file_name = ... # Write the path to the dataset file
df = pd.read_excel(data_file_name)
display(df)
```

(b) Calculate concentrations

Calculate the molar concentration of the two proteins in all samples, the light path for every measurement is 0.1 cm.

Always a good idea to assign known values to variables

```
protein_1_ext_coeff = 180000
protein_2_ext_coeff = 80000
l = 0.1
```

> Tip
>
> You can set new columns in a `DataFrame` by just assigning to it
>
> ```
> df['new_column'] = [1, 2, 3, ..., 42]
> ```
>
> It can also be set as a computation of a property from another row
>
> ```
> df['new_column'] = df['current_column'] / 4
> ```

```
df['protein1_healthy_molar_conc'] = ... # Calculate for concentration in healthy for
protein 1.
... # Your code that updates the data frame with the 3 other new columns.
display(df)
```

(c). Concentrations in mg/mL

Add another set of four columns containing the concentrations in mg/mL.

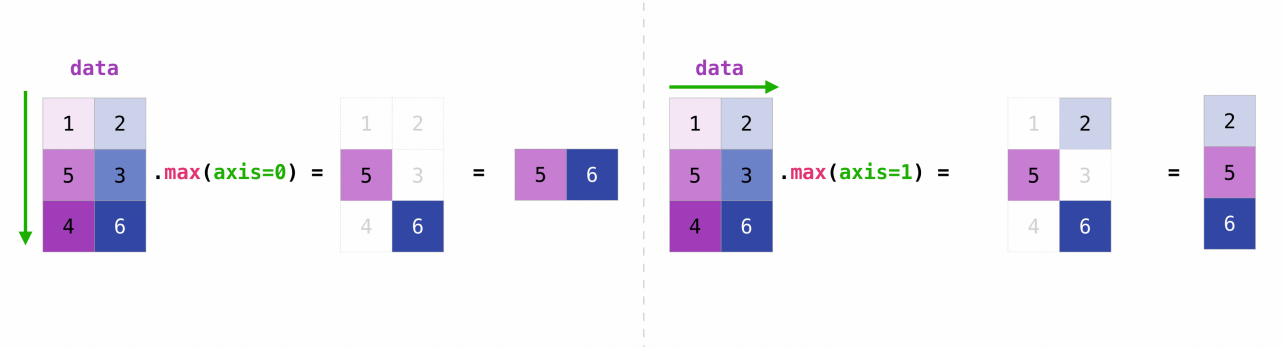
```
protein_1_mw = 130 * 10**3
protein_2_mw = 57 * 10**3
```

```
df['protein1_healthy_conc'] = ...
df['protein1_patient_conc'] = ...
df['protein2_healthy_conc'] = ...
df['protein2_patient_conc'] = ...

names = ['protein1_healthy_conc', 'protein1_patient_conc', 'protein2_healthy_conc',
'protein2_patient_conc']
display(df[names])
```

(d) Mean concentration

Now that we have the mass concentrations, calculate the mean mass concentration in the four categories.

> Tip
>
> When displaying the dataframe above we indexed it with `names` as `df[names]`. We can do the same to compute something over just the four rows.
>
> For example the if we have a `DataFrame` called `example_df`, we can calculate the mean over the **rows** as:
>
> ```
> example_df[names].mean(axis=0)
> ```
>
> Here `axis=0` means that we apply the operation over the first axis which by convention are the rows. The figure below visualizes this
>
> 
>

```
mean = ...
unit_mean = "..."
print(mean)
print("unit_mean")
```

(e) Standard deviation

Calculate the standard deviation

> Tip
>
> The standard deviation can be calculated using the `.std`-method that works in the same way as the `.mean`-method we used above.

```
std = ...
unit_std = "..."
print(std)
print("unit_std")
```

(f) Analyze the results

Consider the following questions

- By comparing healthy individuals with patients, could any of the two proteins would be involved in disease development or be used as a biomarker for diagnosing the disease?
- What additional information does the standard deviation provide in addition to the average value of the protein concentration?

---

5 Extinction coefficient of human myoglobin

The protein of human myoglobin is given below

```
sequence = """GLSDGEWQLVLNVWGKVEADIPGHGQEVLIRLFKGHPETLEKFDKFKHLKSEDEMKASEDLKKHGA
TVLTALGGILKKKGHHEAEIKPLAQSHATKHKIPVKYLEFISECIIQVLQSKHPGDFGADAQGAMNKALELFRKDMASNY
KELGFQG"""
```

We want to calculate the *extinction coefficient* of this protein, we have seen that this can be calculated using the formula

$$\epsilon(280\text{nm}) = N_{Trp}\epsilon_{Trp} + N_{Tyr}\epsilon_{Tyr} + N_{Cys}\epsilon_{Cys} \tag{1}$$

Where N_{Trp} is the number of Tryptophan in the protein (and likewise for the other two terms), and the three constants A, B and C are given as

$$\epsilon_{Trp} = 5500\ \text{M}^{-1}\text{cm}^{-1}$$
$$\epsilon_{Tyr} = 1490\ \text{M}^{-1}\text{cm}^{-1}$$
$$\epsilon_{Cys} = 125\ \text{M}^{-1}\text{cm}^{-1}$$

In order to calculate the formula we need to know the count of the relevant residues, we can use Python to get that - for example we can count the number of Tryptophan like so;

```
N_trp = sequence.count("W")
```

(a) Determine the number of residues

In the cell below find the number of residues

```
N_tyr = ... # Your code here
... # Your code here for the N_cys.
```

You can check what Python has stored each variable by using `print`

```
print(N_trp)
print(f"{N_tyr = }") # This is just a way of make a string that looks nice.
print(f"{N_cys = }")
```

```
2
N_tyr = 2
N_cys = 1
```

(b) Calculate the extinction coefficient

Use equation (Equation 1) to calculate the extinction coefficient of human myoglobin.

```
eps_trp = 5500
eps_tyr = 1490
eps_cys = 125
```

```
epsilon = ... # Erstat ... med din kode.
unit_epsilon = "..."
print(f"{epsilon = :.0f} {unit_epsilon}")
```

What are the units of this value?

(c) Compare residue count to ProtParam

ProtParam is an online tool that calculates various physical and chemical parameters from a given protein sequence and is used worldwide in research laboratories.

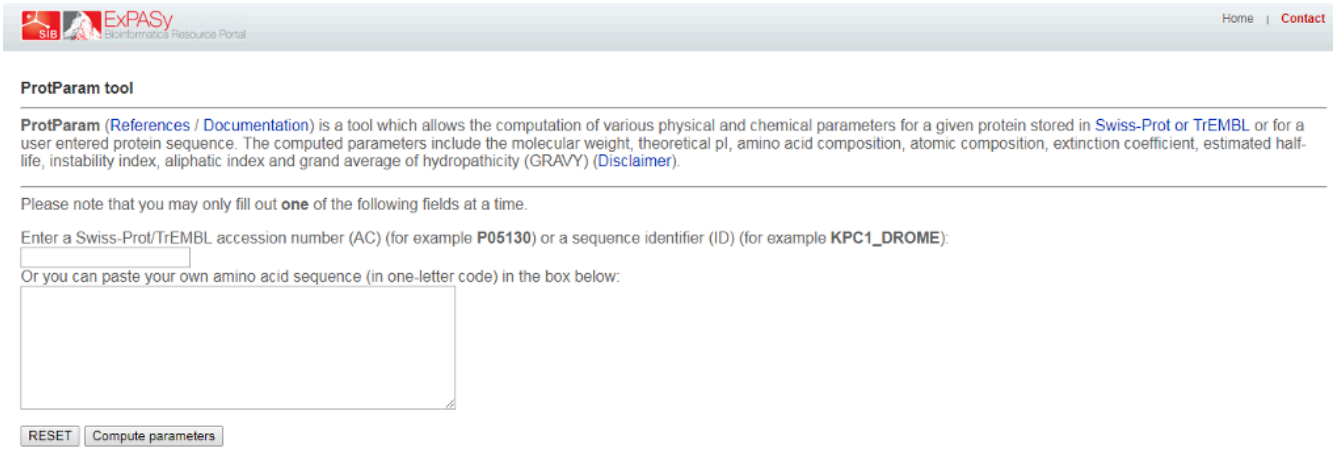


Figure 2: Protpram tool input page.

Go to ProtPram at this link: https://web.expasy.org/protparam/ and paste the sequence and click **Compute Parameters**. You should then see the calculated parameters, similar to in the image below

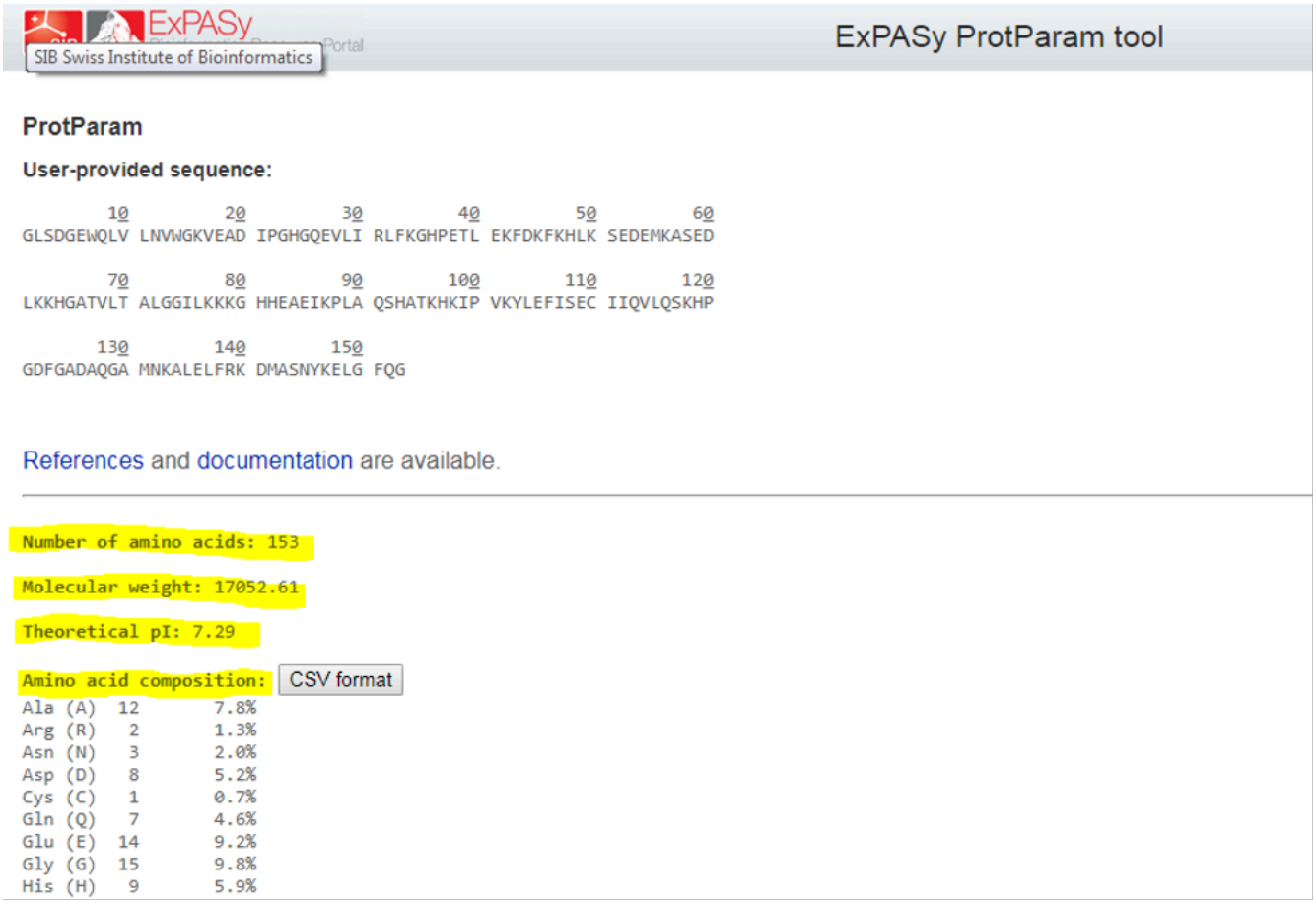


Figure 3: Protpram tool calculated properties.

On the output page you will see a calculated extinction coefficient. Does that match your calculation? If not, what is the different between the assumptions made?

(d) Calculate the absorbance

Using the extinction coefficient and the molecular weight given by ProtParam, calculate the absorbance at 280 nm of a myoglobin solution at a concentration of 1 mg/mL in a cuvette with a light path of 1 cm.

```
molecular_weight = ... # Find the value on ProtPram (It has units of g/mol)
path_length = ... # Set the value of the path length
concentration = ... # Set the value of the concentraiton.
```

Remember to convert the concentration to mol/L.

```
A280 = ...
print(A280)
```

This value is what is known as the A280(0.1%) of a protein, i.e. the absorbance of a given protein at a concentration of 0.1% weight/volume (= 1 g/L = 1 mg/mL).

(e) Titin

We have now calculated the extinction coefficient of a protein, now we will make our code more reusable so that it can be applied to other proteins easily.

> **Note**
>
> A function in Python is a set of instructions, like a recipe, that can be defined and reused multiple times. The syntax is like this
>
> ```
> 1 def my_function(input_1, input_2):
> 2     # Some operation
> 3     some_internal_variable = input_1 + input_2
> 4     ...
> 5     return something_weve_calculated
> ```
>
> **Line 1**
>
> The `def` command is used to define the functions name, here `my_function`, and state its inputs, e.g. the name and ingredients of a recipe.
>
> **Line 3**
>
> The body of the function defines what it does, the body can have any number of lines of code. Similar to the instructions of a recipe.
>
> **Line 5**
>
> Finally, the function can `return` something, like the final product of a recipe.
>
> Note that the function is not executed by doing this, like how a cake isn't baked by writing down the recipe, in order actually use the function it needs to be *called*
>
> ```
> output = my_function(1, 2)
> ```
>
> This is also how we have already used other functions like `print`.

The way of doing so is by defining a function that does the necessary operations for a given sequence. In this way the code can be reused for any sequence.

Finish implementing the body of the function below, note that you have already written all the required code - you just need to copy it into the function.

```
def extinction_coefficient(sequence):

    # Start by counting
    ...

    # Define the residue extinction coeffiecients
    eps_trp = 5500
    eps_tyr = 1490
    eps_cys = 125

    # Calculate
```

```
    epsilon = ...
    unit_epsilon = "..."

    print(f"{epsilon = :.0f} {unit_epsilon}")
```

It's always a good idea to check that functions do what we expect, so we can confirm that it gives the same result for human myoglobin as we calculated before

```
epsilon_func = extinction_coefficient(sequence)
print(f"Epsilon: {epsilon}")
print(f"Epsilon from function: {epsilon_func}")
```

The largest known protein is Titin, the cell below loads the sequence of titin and prints a few bits of information about it. (You can also find the full sequence in the dataset `extin-coeff-human-myogl.txt`.)

```
from fysisk_biokemi.datasets import load_dataset
titin_sequence = load_dataset('titin')

print(f'Titin number of residues: {len(titin_sequence)}')
print('First 100 residues')
print(titin_sequence[0:100])
```

Use your function to calculate the extinction coefficient of titin.

```
titin_eps = ... # Your code here.
print(titin_eps)
```

> **Tip**
>
> You don't need to understand the code below, it's just ment to illustrate that knowing some Python will allow you to explore the topics that interest you in more detail.

In general Python is very powerful at letting us explore properties of sequences, for example the cell below calculates number of residues between each Tryptophan in the Titin sequence and plot the distribution.

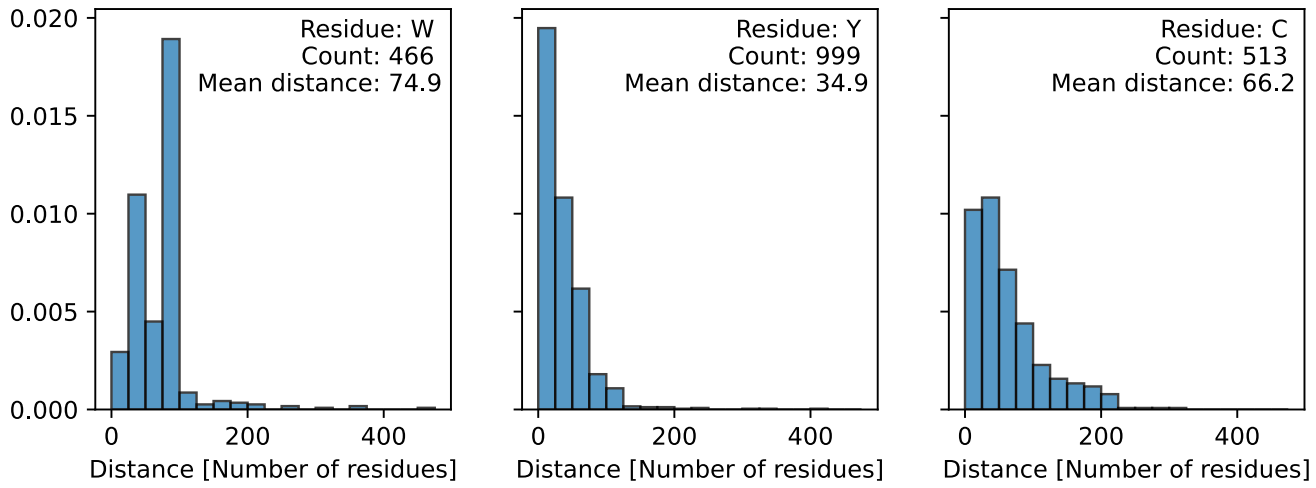
```
import numpy as np
import matplotlib.pyplot as plt

def get_distance(sequence, letter):
    W_index = np.argwhere(np.array([l for l in sequence]) == letter)
    count = len(W_index)
    distance = (W_index - np.roll(W_index, 1))[1:]
    return distance, count

letters = ['W', 'Y', 'C']
```

```
fig, axes = plt.subplots(1, 3, figsize=(3*3, 3), sharey=True)
axes = axes.flatten()

for ax, letter in zip(axes, letters):
    distance, count = get_distance(titin_sequence, letter)
    ax.hist(distance, bins=np.arange(0, 500, 25),
        edgecolor='black', alpha=0.75, density=True)
    ax.set_xlabel('Distance [Number of residues]')
    info = f'Residue: {letter} \nCount: {count} \nMean distance: {np.mean(distance):.1f}'
    ax.text(0.975, 0.975, info, transform=ax.transAxes, ha='right', va='top')
```



---

6 Tryptophan absorption and fluorescence

```
from fysisk_biokemi.datasets import load_dataset
from IPython.display import display
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
pd.set_option('display.max_rows', 6)
```

You investigate a protein that neither contains tyrosine nor cysteine residues. A 37 µM solution gives an absorbance of 0.41 at 280nm at a light path of 1 cm.

(a) Number of residues

How many tryptophan residues does the protein contain?

```
...
```

(b) Concentration

You would like to conduct a protein stability experiment at an absorbance of 0,8 (path length 1cm). What concentration should you use?

```
c_prot = ...
unit_c_prot = "..."
print(c_prot, unit_c_prot)
```

(c) Plot the spectra

The cell below loads extinction and emission spectra for tryptophan in aqueous buffer. (The data files are `trypt-absor-fluor-emission.xlsx` and `trypt-absor-fluor-extinction.xlsx`)

```
df_emission = load_dataset('week47_1_emi')
df_extinction = load_dataset('week47_1_ext')
display(df_emission)
display(df_extinction)
```

	wavelength_(nm)	emission_(AU)
0	280.0	379
1	280.5	396
2	281.0	403
...	...	...
396	478.0	5024
397	478.5	4950
398	479.0	4813

	wavelength_(nm)	molar_extinction_(cm-1/M)
0	219.75	12488
1	220.00	12872
2	220.25	12428
...	...	...
396	318.75	14
397	319.00	18
398	319.25	45

Make plot showing the two spectra

```
fig, axes = plt.subplots(1, 2, figsize=(7, 3), layout='constrained')

# Emission
ax = axes[0]
ax.set_title('Emission')

# Replace ... with your code.
```

```
ax.plot(..., ..., label='Emission', color='C0')

ax.set_ylabel('Emission')
ax.set_xlabel('Wavelength [nm]')

# Extinction
ax = axes[1]
ax.set_title('Extinction')

# Replace ... to plot the extinction spectrum
...

# Customization
ax.set_ylabel('Extinction')
ax.set_xlabel('Wavelength [nm]')
plt.show()
```

(d) Stokes shift

From the plot estimate the Stokes shift

```
estimated_stokes_shift = ...
unit_stokes_shift = "..."
print(f"{estimated_stokes_shift = :.0f} {unit_stokes_shift}")
```

Compare your estimated Stokes shift to the output of the cell below that computes the stokes shift.

```
max_index = np.argmax(df_emission['emission_(AU)'])
emission_wavelength_max = df_emission['wavelength_(nm)'][max_index]

# Find maximum extinction between 250 and 300 nm
filtered_extinction = df_extinction[
    (df_extinction['wavelength_(nm)'] >= 250) &
    (df_extinction['wavelength_(nm)'] <= 300)
]

max_index = filtered_extinction['molar_extinction_(cm-1/M)'].idxmax()
extinction_wavelength_max = df_extinction.loc[max_index, 'wavelength_(nm)']

stokes_shift = emission_wavelength_max - extinction_wavelength_max

print(f"{emission_wavelength_max = :.1f} nm")
print(f"{extinction_wavelength_max = :.1f} nm")
print(f"{stokes_shift = :.1f} nm")
print(f"{estimated_stokes_shift = :.1f} nm")
```

(e) pH and how it changes properties.

Next, you compare the fluorescence emission spectra for your protein at pH 7 and at pH 2. You can find the data in the files `trypt-absor-fluor-ph2.xlsx` and `trypt-absor-fluor-ph7.xlsx` - the cell below also loads these datasets.

The datasets are loaded in a different way than you've seen before - as help to you as this exercise deals with multiple datasets. The datasets could also be loaded by the pd.read_excel('data_file_name') function you used ealier.

```
df_ph2 = load_dataset('week47_1_ph2')
df_ph7 = load_dataset('week47_1_ph7')
display(df_ph2, df_ph2)
```

	Wavelength(nm)	Fluo_Int
0	311.623037	0.890000
1	314.450262	1.128668
2	315.863874	1.128668
...	...	...
43	367.068063	27.990971
44	368.638743	27.313770
45	370.052356	26.636569

	Wavelength(nm)	Fluo_Int
0	311.623037	0.890000
1	314.450262	1.128668
2	315.863874	1.128668
...	...	...
43	367.068063	27.990971
44	368.638743	27.313770
45	370.052356	26.636569

Make a plot comparing the two emission spectra. Which resembles the spectrum of tryptophan in water most?

```
fig, ax = plt.subplots()

ax.plot(..., ..., label='pH = 2') # Replace to plot at pH = 2
... # Replace to plot at pH = 7

ax.set_xlabel('Wavelength [nm]')
ax.set_ylabel('Flouresence [AU]')
ax.legend()
plt.show()
```

(f) Explanation.

Explain why the same protein gives such different spectra at pH 2 and pH 7.

---

7 The fluorescent protein “mCherry”.

```
import pandas as pd
from IPython.display import display
from pathlib import Path
```

The spectra of many fluorescent proteins can be found at the website: www.fpbase.org. Go to FPbase and search for “mCherry”.

(a) Find parameters.

Find the following parameters for the protein

- Extinction coefficient at absorbance maximum
- Quantum yield
- The organism from which it was originally isolated
- Molecular weight

Save them to seperate variables in the cell below.

```
... # Your answers here.
```

(b) Absorbance

What is the absorbance of a 1 µM solution of mCherry at its absorption maximum at a path length of 1 cm?

```
c = ... # Put the concentration in Molar
l = ... # Path length in cm
A_max = ... # Calculate the adsorbance.
print(f"{A_max = :3.3f}")
```

(c) Extinction coefficient from sequence.

> **Note**
>
> We will treat the sequence as a `str`, like any other text, `str`’s are defined like
>
> ```
> one_line_string = "This is some text, that makes up my string."
> ```
>
> Which uses `"`-quotation marks around the text, for longer strings it can be useful to instead use
>
> ```
> multi_line_string = """This is a very long text, so long in fact that it
> takes up multiple lines and I therefore use a slightly different syntax.
> To make the string longer I will confess that I am hungry right now.
> """
> ```

The sequence of the protein is also given. From this determine the extinction coefficient at 280 nm.

Start by taking the sequence from the website and assigning it to the variable `sequence` in the cell below.

```
sequence = """
Put the sequence in here to make a str with the sequence.
Remember to remove this text as it is not part of the sequence.
"""
```

Now use the sequence to calculate the extinction coefficient, finish the code below (or take the function you implemented in a previous exercise!)

```
# This is a "dictionary" with the extinction coefficients of the relevant
# amino acid residues. Dictionaries are indexed with 'keys', so you can retrieve
# the value for W as: ext_residue["W"].
ext_residue = {"W": 5500, "Y": 1490, "C": 125}

# Write code to calculate the extinction coefficent
...
```

(d) Concentration from A280nm

What is the molar concentration of solution of mCherry with an A_280nm of 0.45

```
A280 = ...
c_mol = ...
unit_c_mol = "..."
print(f"{c_mol = :3.3f} {unit_c_mol}")
```

The excitation and emission spectra can be downloaded as a csv-file by clicking the download icon as highlighted below

Figure 4: Screenshot of the download button on fpbase

(e) Load the dataset

Go to www.fpbase.org and download the spectrum as described above.

Load the dataset as a `DataFrame` with pandas. Set `data_file_name` to the name of the downloaded dataset file.

```
data_file_name = ... # Write the name of the dataset file
df = pd.read_csv(data_file_name)
display(df)
```

(f) Plot spectra

Make your own plot showing the excitation and emission spectra of "mCherry" using the above data.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(..., ..., label='Excitation') # Replace ... with your code
... # Replace ... with your code plotting the emission spectrum.
ax.legend()
```

```
ax.set_xlabel('Wavelength')
ax.set_ylabel('Extinction & Emission')
plt.show()
```

(g) Stokes shift

Estimate the Stokes shift of mCherry from the plot.

```
estimated_stokes_shift = ... # Your estimation
unit_stokes_shift = "..."
```

The cell below shows how this could calculated using Python

```
import numpy as np
# Start by extracting the wavelengths from the DataFrame
wavelengths = df['wavelength']

# Find the indices
ex_max_idx = np.argmax(df['mCherry ex'])
em_max_idx = np.argmax(df['mCherry em'])

# Use the index to find the corresponding wavelengths
lambda_ex = wavelengths[ex_max_idx]
lambda_em = wavelengths[em_max_idx]

stokes_shift = lambda_em - lambda_ex

print(f"{lambda_ex = :d}")
print(f"{lambda_em = :d}")
print(f"{stokes_shift = :d}")
print(f"{estimated_stokes_shift = :d}")
```

Tip

If you have two arrays `A` and `B` you can find the entry in `A` corresponding to the largest value in `B` like this

```
A_at_B_max = A[np.argmax(B)]
```

The `np.argmax` stands for **arg**ument **max**imum meaning that it finds the index of the maximum value in a given array. The figure below illustrates this

Figure 5: Illustration of argmax

(h) Colors

Compare to a plot of the visual spectrum. What colors are the light that corresponds to the excitation and emission maxima respectively?

Why do you think the protein is called *Cherry*?

---

8 Python: Introduction to curve fitting

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
```

Curve fitting is a fundamental skill in biochemistry and biophysics for analyzing experimental data. We use curve fitting to determine parameters in mathematical models that describe biological processes like enzyme kinetics, binding affinity, and reaction rates.

The basic idea is to find the parameters of a mathematical function that best describes our experimental data.

(a) Understanding the problem

Real biochemical data is often requires complex fitting functions. Therefore, we will start out with a simpler data that follows a linear relationship. The data below represents a theoretical experiment where we measure some output y at different input values x:

```
## This makes two arrays (columns) ##
## One with the x coordinates ##
## One with the y coordinates ##
x_data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y_data = np.array([2.1, 4.2, 6.0, 7.8, 10.1, 12.2, 13.9, 16.1, 18.0, 20.2])
```

The functions above simply make two arrays - i.e. columns of data - containing the number. You can view the content of these colmns by with `print(x_data)` or `print(y_data)`.

```
fig, ax = plt.subplots()

## Your task: Plot the data as a scatter plot ##
## Recall that you can do this with: ax.plot(x_data, y_data, 'o')
...
```

Looking at this data, we can see it roughly follows a straight line. Let's assume our model is:

$$y = ax + b$$

where a and b are parameters we want to determine from the data.

(b) Defining a fitting function

To use `scipy.optimize.curve_fit`, we need to define a function where:

- The **first** argument is the independent variable (x)
- The remaining arguments are the parameters to be fitted

Define a linear function for fitting:

```
def linear_function(x, a, b):
    result = ...  ## Your task: Write the linear function ##
    return result
```

Always a good idea to check that your function works as expected

As a test we are adding 1 in the place of x, 2 in the place of a and 0 in the place of b. In other words: $1 \cdot 2 + 0 = 2$

```
linear_function(1, 2, 0) # Should give 2
```

When in doubt it is a good idea to do sanity checks like this.

(c) Your first curve fit

Now we can use `curve_fit` to find the best parameters. The basic syntax is:

```
fitted_parameters, trash = curve_fit(function, x_data, y_data, p0=initial_guess)
```

Where:

- `function`: The function we defined above
- `x_data`, `y_data`: Our experimental data
- `p0`: Initial guess for parameters (optional but recommended)
- `fitted_parameters`: The optimized parameters (what we want!)
- `trash`: Don't worry about this.

Finish the cell below to perform the curve fit by adding the three missing arguments to the call to `curve_fit`.

```
# Initial guess: a=2, b=0
initial_guess = [2, 0]

## Your task: Perform the fit by filling out the line of code below ##
## You need give the function three arguments: ##
## 1. The function: linear_function ##
## 2. The x-data: x_data
## 3. The y-data: y_data
fitted_parameters, trash = curve_fit(..., ..., ..., p0=initial_guess)

## Extract parameters ##
a_fit, b_fit = fitted_parameters

print(fitted_parameters)
print(a_fit)
print(b_fit)
```

(d) Visualizing the fit

It's crucial to always plot your fit to see how well it describes the data. To do this we evaluate the function with the fitted parameters and a densely sampled independent variable.

```
## This makes an array with 100 equally spaced points between 0 and 11. ##
x_smooth = np.linspace(0, 11, 100)

## Your task: Calculate linear_function using x_smooth, a_fit and b_fit ##
y_fit = ...
```

Then we can plot it

```
fig, ax = plt.subplots()

## Your task: Plot the fitted function using x_smooth and y_fit ##
ax.plot(..., ...)

## Plots the experimental data ##
ax.plot(x_data, y_data, 'o')
```

(e) Nonlinear fitting: Exponential decay

Many biological processes follow nonlinear relationships. Let's work with exponential decay, which is common in biochemistry (e.g., radioactive decay or unimolecular chemical reactions). Upload the file `exp_decay_data.xlsx` to jupyter. then load the dataset as a `DataFrame` with pandas. Set `data_file_name` to the name of the downloaded dataset file.

```
data_file_name = ... # Write the name of the dataset file
df = pd.read_excel(data_file_name)
display(df)
```

Now make a plot of the data

```
# Generate exponential decay data
fig, ax = plt.subplots()

## Your task: Plot the data from the dataframe ´df´. ##
## Recall that your get the time-column as df['time'] ##
## Similarly you get the signal-column  as df['signal'] ##
ax.plot(..., ...)
```

The model we want to fit is:

$$\text{signal} = Ae^{-kt}$$

Define the exponential decay function:

```
def exponential_decay(t, A, k):
    ## Your task: Calculate the exponential function ##
    ## Remember that np.exp calculates the exponential function ##
    result = ...
    return result

## This calculates the function for t=1, A=1, k=1 ##
## If you did it correctly you should see 0.3678...
test_result = exponential_decay(1, 1, 1)
print(test_result)
```

Now fit the exponential function to the data:

```
## We set the initial guess as A=8 and k=1 ##
initial_guess = [8, 1]

## Your task: Make the fit using the exponenital_decay function, and the time and signal
data.
##
fitted_parameters, trash = curve_fit(..., ..., ..., p0=initial_guess)

# Extract parameters
A_fit, k_fit = fitted_parameters

print(A_fit)
print(k_fit)
```

Again we should plot to check that it looks as expected

```
## This makes 100 linearly spaced points between 0 and 5.
t_smooth = np.linspace(0, 5, 100)

## Your task: Calculate the function using the fitted parameters and t_smooth. ##
signal_fit = ... # Evaluate using the exponential_decay funciton
```

Now we can plot the fit along with the data.

```
fig, ax = plt.subplots()

## Your task: Plot the observations - you can copy this line from the ##
## previous plot. ##
ax.plot(..., ...)

## Your task: Plot the fit using t_smooth and signal_fit
ax.plot(..., ...)
```

Key points for successful curve fitting:

1. **Always plot your data first** to understand what kind of function might fit
2. **Always plot the fit** to visually check if it makes sense
3. **Check if parameters are reasonable** based on your biochemical knowledge

! Summary of `curve_fit`

Fitting refers to finding the parameters that make an assumed functional form best 'fit' the data.

In Python we will use the `curve_fit` from the `scipy` package to do so. The function looks like this

```
curve_fit(function,
            x_data,
            y_data,
            p0=[param_1, param_2, ...])
```

The arguments are

- `function`: A python function where the **first** argument is the independent variable (typically the x coordinate), and other arguments are the parameters of the functions.
- `x_data`: The observed values of the independent variable (The x coordinates).
- `y_data`: The observed values of the dependent variable (The y coordinates) .
- `p0`: Initial guesses for the parameters.

When called `curve_fit` starts by calculating how well the functions fits the data with the initial parameters in `p0` and then iteratively improves the fit by trying new values for the parameters in an intelligent way.

The found parameters will generally depend on `p0` and it is therefore necessary to provide a good (or good enough) guess for `p0`.

Generally, the best way to learn more about a function is to read it's documentation and then play around with it. The documentation is in this case on the SciPy website. You don't need to read it, unless you want more details.

You now have the fundamental skills needed to fit curves to biochemical data! In the exercises, you'll apply these techniques to analyze real experimental data and extract meaningful biological parameters.